

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САМАРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИМЕНИ АКАДЕМИКА С. П. КОРОЛЁВА»
(Самарский университет)

Институт естественных и математических наук
Факультет механико-математический
Кафедра безопасности информационных систем

КУРСОВАЯ РАБОТА

ПРИМЕНЕНИЕ АППАРАТА БУЛЕВЫХ ФУНКЦИЙ
В КРИПТОГРАФИИ

по специальности 10.05.01 Компьютерная безопасность
(уровень специалитета)

Обучающийся 4442-100501D гр. Э. Э. Морозов
Руководитель КР,
доцент, к.ф.-м.н. В. В. Бондаренко
Нормоконтролер 02.06.2025

Самара 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Основные понятия	4
1.1 Модель криптографической системы	4
1.2 Булевы функции в криптографии	8
ВЫВОДЫ	8
2 Булевы функции	9
2.1 Формы представления Булевой функции	9
2.2 Преобразование Мебиуса	13
2.3 Преобразование Уолша-Адамара	16
2.4 Свойства булевых функций	19
2.4.1 Сбалансированность	19
2.4.2 Нелинейность булевых функций	20
2.4.3 Алгебраическая степень	23
2.4.4 Корреляционная иммунность	25
2.5 Производная булевой функции	28
ВЫВОДЫ	30
3 Практическая часть	31
3.1 Программная реализация	32
ВЫВОДЫ	39
ЗАКЛЮЧЕНИЕ	40
СПИСОК СОКРАЩЕННЫХ ОБОЗНАЧЕНИЙ	41
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	42
ПРИЛОЖЕНИЕ А Программная реализация класса булевых функций	43

ВВЕДЕНИЕ

Булевы функции играют ключевую роль в симметричной криптографии [1]. Они используются в качестве комбинаторных и фильтрующих компонентов в генераторах ключевого потока, а также как функции замены в блочных шифрах и других криптографических примитивах. Криптостойкость таких систем напрямую зависит от свойств булевых функций, входящих в их состав.

Для эффективного противодействия различным видам атак – статистическим, линейным, дифференциальным, алгебраическим и другим – булевы функции должны обладать рядом криптографически значимых характеристик: сбалансированностью, высокой нелинейностью, корреляционной иммунностью, устойчивостью и благоприятным дифференциальным поведением. Однако многие из этих требований находятся в противоречии друг с другом, что существенно усложняет задачу конструирования функций с оптимальным набором свойств.

Актуальность темы определяется тем, что выбор подходящих булевых функций критически важен для обеспечения безопасности блочных и потоковых шифров. Поиск таких функций остаётся сложной и востребованной задачей, находящейся в центре внимания современного криптографического анализа.

Целью работы является исследование криптографических свойств булевой функции, в первую очередь, дифференцирования и разработка соответствующего программного инструментария.

Для достижения цели поставлены следующие **задачи**:

- **исследовать** булевы функции и их криптографические свойства;
- **разработать** инструменты для работы с булевыми функциями;
- **оценить** вычислительную сложность используемых алгоритмов.

1 Основные понятия

1.1 Модель криптографической системы

Криптография (в переводе с греческого – «тайнопись») представляет собой искусство и науку сокрытия информации, главная задача которой – обеспечение конфиденциальности данных.

Отправитель – это субъект, заинтересованный в защищённой передаче информации другому субъекту. Под защищённой передачей понимается такая передача, при которой перехват сообщения злоумышленником не даёт ему возможности извлечь содержащуюся в нём информацию. Получатель – субъект, принимающий сообщение от отправителя и способный извлечь из него информацию.

Для защиты содержимого исходное сообщение, называемое открытым текстом, подвергается преобразованию или замене, чтобы затруднить его понимание посторонними лицами. Этот процесс называется шифрованием, а результат преобразования – шифротекстом. Обратный процесс – дешифрование, то есть восстановление исходного текста из зашифрованного. Схема безопасной передачи информации представлена на рисунке 1.



Рисунок 1 – Процесс безопасной передачи информации

Наряду с криптографией – наукой о шифровании информации, существует криптоанализ, который служит для взламывания шифротекстов. Раздел математики, включающий криптографию и криптоанализ, называется криптология.

Будем обозначать открытый текст как M (от *message*, сообщение), а шифротекст как C (от *ciphertext*, зашифрованный текст). M и C будем пред-

ставлять как конечную двоичную последовательность. Для шифрования нам потребуется функция E (от *encrypt*, зашифровать), преобразующая открытый текст в шифротекст и аналогичная ей функция дешифрования D (от *decrypt*, расшифровать), выполняющая преобразования шифротекста в открытый текст:

$$E(M) = C, \quad D(C) = M.$$

Для того, чтобы расшифрованный шифротекст соответствовал изначальному открытому тексту, потребуем выполнение:

$$D(E(M)) = M.$$

Тогда под шифром или криптографическим алгоритмом будем понимать совокупность двух функций E и D . Выделяют ограниченные криптографические алгоритмы и криптографические алгоритмы с использованием ключа.

Под ограниченными алгоритмами понимают шифры, безопасность которых основывается лишь на секретности самого алгоритма шифрования. Такой класс шифров в настоящее время может представлять лишь исторический интерес, так как его использование сопровождается существенными ограничениями. Например, в случае выхода человека из группы, использующей шифр данного класса, необходимо менять алгоритм шифрования. Отсутствие стандартизации алгоритма не даёт пользователям, использующим данный шифр, но не знакомым с его сутью, уверенности в безопасности передачи данных.

В современном мире эти проблемы решаются благодаря использованию ключа шифрования K , который параметризует функции шифрования и расшифрования. Таким образом, уравнения шифрования и расшифрования, а также условие на соответствие расшифрованного текста изначальному со-

общению при использовании ключа будут выглядеть следующим образом:

$$E_K(M) = C, \quad D_K(C) = M, \quad D_k(E_K(M)) = M.$$

В свою очередь шифры с использованием ключа делят на симметричные и асимметричные. Симметричные алгоритмы используют одинаковый или зависящие друг от друга ключи. Асимметричные используют различные ключи для функций шифрования и расшифрования, которые имеют следующий вид:

$$E_{K_1}(M) = C, \quad D_{K_2}(C) = M, \quad D_{K_2}(E_{K_1}(M)) = M.$$

Криптосистема – совокупность алгоритма шифрования, множества возможных открытых текстов и шифротекстов, множество возможных ключей. В общем виде процесс передачи информации с использованием криптосистемы можно представить как на рисунке 2.



Рисунок 2 – Передача информации по криптографически защищённому каналу

Симметричные и асимметричные шифры используют для разных целей. Симметричные обладают более высокой скоростью при работе с большим количеством данных, асимметричные шифры полезны для безопасного обмена ключами и цифровыми подписями. Современные симметричные шифры подразделяются на потоковые и блочные [2]. Критерием классификации является способ обработки открытого текста. Если шифр разбивает исходное сообщение на блоки фиксированной длины, например, несколь-

ко байт, и шифрует каждый блок как единое целое, то он называется блочным. В случае же, когда шифрование осуществляется посимвольно, например, несколько бит или байт, и каждый элемент текста шифруется последовательно, говорят о потоковом шифре.

Ключевым компонентом многих блочных шифров является *S*-блок (*substitution box*) – нелинейная функция, преобразующая входные биты в выходные. Поведение *S*-блока часто моделируется в виде набора булевых функций, описывающих выходные биты как функции от входных. Качество *S*-блока, а значит и безопасность шифра, напрямую зависит от криптографических свойств этих булевых функций – таких как нелинейность, сбалансированность, устойчивость к линейному и дифференциальному криптоанализу.

Потоковые шифры используют псевдослучайную последовательность битов (ключевой поток), которая затем смешивается с открытым текстом (обычно с помощью операции *XOR*). Генерация ключевого потока может основываться на регистрах сдвига с линейной обратной связью (*LFSR*) [3], нелинейной функцией обновления, булевыми фильтрами и комбинационными функциями. В этом контексте булевые функции также играют центральную роль. Например, в фильтрующих потоковых шифрах нелинейная булева функция используется для фильтрации состояния регистра и формирования выходного бита. В комбинационных потоковых шифрах выходной бит формируется как значение булевой функции от состояний нескольких регистров. Таким образом, булевые функции являются фундаментом симметричной криптографии, обеспечивая нелинейность и стойкость к различным атакам. Исследование их свойств – таких как спектр Уолша, алгебраическая степень, сбалансированность, корреляционная иммунность и вес Хэмминга – позволяет оценивать и проектировать криптографические примитивы с заданным уровнем стойкости.

1.2 Булевы функции в криптографии

Булевой функцией от n переменных называется функция $f : \mathbb{Z}_2^n \rightarrow \mathbb{Z}_2$. Множество всех булевых функций от n переменных обозначим через $\mathcal{F}_n$. Нетрудно убедиться, что $|\mathcal{F}_n| = 2^{2^n}$.

Сложность и непредсказуемость поведения булевых функций делают их важным объектом исследования в контексте криптоанализа. В блочных шифрах основным (а зачастую и единственным) нелинейным криптографическим примитивом является таблица замен — так называемая *S-box* (от англ. *Substitution-box*), представляющая собой отображение из n -битных входов в m -битные выходы. Чаще всего используется случай, когда $n = m$, при этом типичными значениями являются $n = m = 4, 6, 8$, что обусловлено как теоретическими соображениями криптостойкости, так и практическими аспектами реализации на аппаратном или программном уровне.

Таблица подстановок (*S-box*) часто реализуется в виде перестановки n -битных входных значений. Такую перестановку можно выразить с использованием системы булевых функций: каждое выходное значение *S-box* представляется как набор из n булевых функций, каждая из которых отображает n -битный вход в один бит выхода. Для обеспечения криптографической стойкости эти булевы функции должны обладать такими свойствами, как сбалансированность, нелинейность, высокая алгебраическая степень и корреляционная иммунность.

ВЫВОДЫ

В первом разделе проведён обзор основных понятий криптографии, представлен общий вид криптографической модели, а также подчеркнута важность булевых функций в построении современных шифров. В следующем разделе подробно рассматривается природа булевых функций, их основные свойства.

2 Булевы функции

2.1 Формы представления Булевой функции

Для работы с булевыми функциями используются следующие формы представления [4].

Таблица истинности - это таблица размером $2^n \times (n + 1)$ для однозначного представления булевой функции от n переменных, в которой каждая строка соответствует одному из всех возможных наборов значений аргументов булевой функции, упорядоченных лексикографически; в последнем столбце указывается значение функции на соответствующем наборе.

Также выделяется *Таблицу истинности полярности*, которая отличается от таблицы истинности лишь последним столбцом: вместо значений функции, в последнем столбце указывается значение функции $(-1)^f = 1 - 2f$, где, если $f(x) = 0$, то $(-1)^f = 1$, а если $f(x) = 1$, то $(-1)^f = -1$. Таблица истинности может использоваться в качестве промежуточного этапа в преобразовании Уолша-Адамара.

Носитель булевой функции - это множество аргументов, на которых функция принимает единичное значение.

$$\text{supp}(f) = \{x \in \mathbb{Z}_2^n \mid f(x) = 1\}$$

Весом булевой функции называется количество наборов аргументов, на которых функция принимает значение единица, то есть

$$w(f) = \sum_{x \in \mathbb{Z}_2^n} f(x) = |\{x \in \mathbb{Z}_2^n : f(x) = 1\}|.$$

Таким образом, мощность носителя можно оценить как:

$$|\text{supp}(f)| = w(f).$$

Носитель булевой функции может использоваться для анализа свойств нелинейности и сбалансированности.

Аналитические формы представления основаны на использовании логических операций: отрицания ($\neg$), конъюнкции ($\wedge$), дизъюнкции ($\vee$) и исключающего "или" ($\oplus$). Эти формы позволяют выразить функцию в виде логического выражения, составленного из переменных и указанных операций.

Определим степень булевой величины следующим образом: $a^0 = 1$ и $a^1 = a$. Тогда выражение a^b можно выразить как: $(a \oplus 1) \wedge b \oplus 1$ или $a \vee \neg b$. Возведение в степень булевого вектора выполняется поэлементно, а затем результаты объединяются с помощью конъюнкции:

$$a^b = \bigwedge_{i=1}^n a_i^{b_i} = \bigwedge_{i=1}^n a_i \vee \neg b_i.$$

Совершенной дизъюнктивной нормальной формой (СДНФ) булевой функции f называется дизъюнкция всех элементарных конъюнкций, на которых функция принимает значение 1.

$$f(x) = \bigvee_{a \in f^{-1}(1)} \bigwedge_{i=1}^n x_i \oplus \neg a_i = \bigvee_{a \in f^{-1}(1)} \bigwedge_{i=1}^n \neg(x_i \oplus a_i)$$

В свою очередь, *совершенной конъюнктивной нормальной формой (СКНФ)* булевой функции f называется конъюнкция всех элементарных дизъюнкций, на которых функция принимает значение 0.

$$f(x) = \bigwedge_{a \in f^{-1}(0)} \bigvee_{i=1}^n x_i \oplus a_i$$

Совершенные дизъюнктивные и конъюнктивные нормальные формы (СДНФ и СКНФ) представляют собой канонические способы задания булевых функций, однако в криптографии они практически не применяются. Это связано с их экспоненциальной сложностью при увеличении числа пе-

ременных, а также с низкой аналитической пригодностью: такие формы не позволяют эффективно анализировать важные криптографические свойства функций, такие как нелинейность, корреляционная устойчивость и характеристики распространения. Кроме того, СДНФ и СКНФ чувствительны к малейшим изменениям значений функции, что делает их непрактичными при проектировании криптографических примитивов. В связи с этим в криптографической практике предпочтение отдается более компактным и аналитически удобным представлениям, таким как алгебраическая нормальная форма и спектральные методы.

Для следующей формы представления нам потребуются следующие свойства Булевой функции.

1) Разложение Шеннона:

$$f(x_1, \dots, x_n) = \neg x_1 f(0, x_2, \dots, x_n) \vee x_1 f(1, x_2, \dots, x_n);$$

$$2) \neg x = x \oplus 1;$$

$$3) x \vee y = x \oplus y \oplus xy.$$

Согласно 2 и 3 свойству, разложение Шеннона можно переписать следующим образом:

$$f(x_1, \dots, x_n) = (x_1 \oplus 0 \oplus 1)f(0, x_2, \dots, x_n) \vee (x_1 \oplus 1 \oplus 1)f(1, x_2, \dots, x_n).$$

Подобным образом можно провести разложение булевой функции по любому количеству переменных.

Утверждение 1. (о разложении по переменным). Пусть $f \in B_n, m \leq n$. Тогда

$$f(x_1, \dots, x_n) = \bigoplus_{a_1, \dots, a_m \in \mathbb{Z}_2^m} (x_1 \oplus a_1 \oplus 1) \cdot \dots \times \\ \times (x_m \oplus a_m \oplus 1) f(a_1, \dots, a_m, x_{m+1}, \dots, x_n). \quad (1)$$

Доказательство. В силу условия $x_i \oplus a_i \oplus 1 = 1 \Leftrightarrow x_i = a_i$ в сумме останутся лишь слагаемые, для которых $a_i = x_i$, таким образом, правая и левая части равенства будут идентичны. $\square$

Функции $f(a_1, \dots, a_m, x_{m+1}, \dots, x_n)$ из формулы (1) называются *коэффициентами разложения функции f по переменным $x_1, \dots, x_m$* .

Утверждение 2. Пусть $f_1, \dots, f_{2^m}$ - все коэффициенты разложения функции f по некоторым m переменным. Тогда

$$w(f) = \sum_{i=1}^{2^m} w(f_i).$$

Если записать разложение функции f по всем переменным, то получим представление, называемое *совершенной алгебраической нормальной формой* функции f .

$$\begin{aligned} f(x_1, \dots, x_n) &= \bigoplus_{a_1, \dots, a_n \in \mathbb{Z}_2^n} (x_1 \oplus a_1 \oplus 1) \cdot \dots \cdot (x_n \oplus a_n \oplus 1) f(a_1, \dots, a_n) = \\ &= \bigoplus_{\substack{a_1, \dots, a_n \in \mathbb{Z}_2^n \\ f(a_1, \dots, a_n) = 1}} (x_1 \oplus a_1 \oplus 1) \cdot \dots \cdot (x_n \oplus a_n \oplus 1). \end{aligned} \quad (2)$$

Если раскрыть скобки и привести подобные слагаемые ($u \oplus u = 0$), то получится *алгебраическая нормальная форма (АНФ) или полином Жегалкина* функции f - сумма различных слагаемых (мономов) вида $x_1^{\tilde{a}_1} x_2^{\tilde{a}_2} \dots x_n^{\tilde{a}_n}$, где $\tilde{a}_i \in \mathbb{Z}_2$.

Доказательство единственности представления Булевой функции в виде АНФ сводится к сравнению мощностей множества $\mathcal{F}_n$ и множества всевозможных сумм мономов вида $x_1^{\tilde{a}_1} x_2^{\tilde{a}_2} \dots x_n^{\tilde{a}_n}$. Поскольку каждый моном соответствует подмножеству множества переменных $\{x_1, \dots, x_n\}$, всего таких мономов существует 2^n . Каждое подмножество мономов порождает уникальную

сумму, а значит, общее число таких сумм равно 2^{2^n} . Так как мощность множества $\mathcal{F}_n$ также равна 2^{2^n} , и каждой булевой функции соответствует ровно одно выражение в виде суммы мономов по модулю 2, представление в виде АНФ существует и является единственным.

2.2 Преобразование Мебиуса

Введем функцию $g(a_1, \dots, a_n)$, зависящую от булевой функции f , такую, что она принимает значение 1 тогда и только тогда, когда моном $x_1^{a_1} x_2^{a_2} \dots x_n^{a_n}$ входит в АНФ функции f в качестве слагаемого. Тогда справедливо следующее представление:

$$f(x_1, \dots, x_n) = \bigoplus_{a_1, \dots, a_n \in \mathbb{Z}_2^n} g(a_1, \dots, a_n) x_1^{a_1} x_2^{a_2} \dots x_n^{a_n} \quad (3)$$

Значения функции g называются *коэффициентами* АНФ функции f .
Отображение

$$\mu : \mathcal{F}_n \rightarrow \mathbb{Z}_2, \quad \mu(f) = g,$$

при котором каждой булевой функции f сопоставляется её вектор коэффициентов g , называется *преобразованием Мебиуса*.

Пример 1. Пусть функция $f(x_1, x_2, x_3)$ задана следующей таблицей истинности:

x_1	x_2	x_3	f
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

По формуле (2) построим совершенную алгебраическую нормальную форму функции f , а затем путем приведения подобных слагаемых получим АНФ:

$$\begin{aligned} f(x_1, x_2, x_3) &= (x_1 \oplus 1)(x_2 \oplus 1)x_3 \oplus x_1(x_2 \oplus 1)(x_3 \oplus 1) \oplus x_1x_2(x_3 \oplus 1) \oplus \\ &\oplus x_1x_2x_3 = (x_1x_2x_3 \oplus x_1x_3 \oplus x_2x_3 \oplus x_3) \oplus (x_1x_2x_3 \oplus x_1x_2 \oplus x_1x_3 \oplus x_1) \oplus \\ &\oplus (x_1x_2x_3 \oplus x_1x_2) \oplus x_1x_2x_3 = x_2x_3 \oplus x_3 \oplus x_1 \end{aligned}$$

Получили следующее преобразование Мебиуса функции f :

a_1	a_2	a_3	$\mu(f)$
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	0

Очевидно, что способы вычислений, представленный выше является трудоемким и труднореализуемым. Получим более удобную формулу для $\mu(f)$.

Введем частичный порядок на множестве B_n . Пусть $a, b \in B_n$, тогда $a \leq b \Leftrightarrow a_i \leq b_i, \forall i \in [1, n]$. Булева функция f называется *монотонной*, если из $a \leq b$ следует $f(a) \leq f(b)$.

Не трудно заметить, что $x_i^{a_i} = 1 \Leftrightarrow a_i \leq x_i$. Действительно:

$$\begin{aligned} 0^0 &= 1; & 0^1 &= 0; \\ 1^0 &= 1; & 1^1 &= 1. \end{aligned}$$

Для краткости обозначим $x = x_1 \dots x_n$, $a = a_1 \dots a_n$, $x^a = x_1^{a_1} \dots x_n^{a_n}$. Тогда $x^a = 1 \Leftrightarrow a \leq x$, т.е. $a_i \leq x_i \quad \forall i \in [1, n]$. Таким образом, (3) можно

переписать следующим образом:

$$f(x) = \bigoplus_{a \in \mathbb{Z}_2^n} g(a)x^a = \bigoplus_{a \leq x} g(a). \quad (4)$$

Формула (4) позволяет вычислять значения функции f по ее коэффициентам АНФ. Например, для функции f из примера 1 верно обращение формулы (4):

$$f(0, 0, 1) = g(0, 0, 0) \oplus g(0, 0, 1) = 0 \oplus 1 = 1;$$

$$f(0, 1, 0) = g(0, 0, 0) \oplus g(0, 1, 0) = 0 \oplus 0 = 0;$$

$$f(0, 1, 1) = g(0, 0, 0) \oplus g(0, 0, 1) \oplus g(0, 1, 0) \oplus g(0, 1, 1) = 0 \oplus 1 \oplus 0 \oplus 1 = 0;$$

...

Утверждение 3. (о преобразовании Мебиуса). Пусть $f \in \mathcal{F}_n$, $g = \mu(f)$. Тогда для любого $a \in \mathbb{Z}_2^n$ справедливо следующее:

$$g(a) = \bigoplus_{x \leq a} f(x). \quad (5)$$

Доказательство. ММИ по весу вектора a .

База индукции. $g(0^n) = f(0^n)$, где 0^n - это вектор длины n с весом 0.

Действительно, по (4):

$$f(0^n) = \bigoplus_{a \leq 0^n} g(a) = g(0^n).$$

Предположение индукции. Пусть утверждение верно для всех векторов a веса меньше, чем p .

Шаг индукции. Докажем утверждение для вектора a веса p . По формуле (4) и по предположению индукции запишем:

$$f(a) = \bigoplus_{x \leq a} g(x) = \bigoplus_{x < a} g(x) \oplus g(a) = \bigoplus_{x < a} \bigoplus_{y \leq x} f(y) \oplus g(a).$$

Обозначим двойную сумму в последней части равенства через S и рассмотрим её отдельно.

$$S = \bigoplus_{x < a} \bigoplus_{y \leq x} f(y) = \bigoplus_{y < a} f(y) \bigoplus_{y \leq x < a} 1 = \bigoplus_{y < a} f(y).$$

Последнее равенство имеет силу, потому что существует ровно $(2^{w(a)-w(y)} - 1)$, вычитание единицы, потому что $x < a$ - строгое неравенство) - нечетное число таких x , для которых выполнялось условие $y \leq x < a$. Из последней части равенства $f(a)$ выразим $g(a)$:

$$g(a) = S \oplus f(a) = \bigoplus_{y < a} f(y) \oplus f(a) = \bigoplus_{y \leq a} f(y),$$

Что и требовалось доказать. □

Следствие 1.

$$\mu(\mu(f)) = f.$$

Доказательство.

$$f(x) = \bigoplus_{a \leq x} g(a) = \bigoplus_{a \leq x} \bigoplus_{x \leq a} f(x) = f(x)$$

□

2.3 Преобразование Уолша-Адамара

Определение 1. Расстоянием Хэмминга $d(f, g)$ между функциями $f, g \in \mathcal{F}_n$ называется количество наборов аргументов, на которых эти функции принимают различные значения:

$$d(f, g) = |\{x \in \mathbb{Z}_2^n : f(x) \neq g(x)\}| = w(f \oplus g). \quad (6)$$

Определение 2. Скалярным произведением булевых величин $a, x \in \mathbb{Z}_2^n$ называется следующая сумма:

$$(a, x) = \bigoplus_{i=1}^n x_i y_i.$$

Функция (a, x) называется линейной. Множество линейных функций от n переменных обозначается $\mathcal{L}(n)$:

$$\mathcal{L}(n) = \{(a, x) : a \in \mathbb{Z}_2^n\}.$$

Функция степени 0 или 1 называется *аффинной*; множество всех аффинных функций от n переменных обозначается $\mathcal{A}(n)$:

$$\mathcal{A} = \{a_0(a, x) : a_0 \in \mathbb{Z}_2, a \in \mathbb{Z}_2^n\}$$

Определение 3. Преобразование Уолша-Адамара (ПУА) функция $f \in \mathcal{F}_n$ называется функцией $\hat{f} : \mathbb{Z}_2^n \rightarrow \mathbb{Z}$:

$$\hat{f}(a) = \sum_{x \in \mathbb{Z}_2^n} (-1)^{f(x) \oplus (a, x)}.$$

Значения, полученные с помощью ПУА, называются коэффициентами ПУА.

Рассмотрим поподробнее слагаемое ПУА и заметим, что:

$$(-1)^{f(x) \oplus (a, x)} = \begin{cases} 1, & \text{если } f(x) = (a, x) \\ -1, & \text{если } f(x) \neq (a, x) \end{cases}$$

Исходя из этого, $\hat{f}$ — представляет из себя разность количества совпадений (σ) и несовпадений (δ) функций $f(x)$, (a, x) . Так как x может принимать 2^n значений, выразим σ через δ : $\sigma = 2^n - \delta$. Очевидно, что $\delta = d(f, (a, x))$.

Используя замены, получим:

$$\hat{f}(a) = 2^n - 2d(f, (a, x)).$$

Определение 4. *Спектром булевой функции называется множество модулей коэффициентов ПУА.*

Пример 2. *Найдём коэффициенты ПУА и спектр функции XOR:*

$$\hat{f}(0, 0) = 2^2 - 2d(f, ((0, 0), x)) = 4 - 2 \cdot 2 = 0$$

$$\hat{f}(0, 1) = 2^2 - 2d(f, ((0, 1), x)) = 4 - 2 \cdot 2 = 0$$

$$\hat{f}(1, 0) = 2^2 - 2d(f, ((1, 0), x)) = 4 - 2 \cdot 2 = 0$$

$$\hat{f}(1, 1) = 2^2 - 2d(f, ((1, 1), x)) = 4 - 2 \cdot 0 = 4$$

x_1	x_2	$f(x)$	$\hat{f}(x)$
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	4

Определение 5. *Матрицей Адамара порядка k называют квадратную матрицу размера $k \times k$, составленную из элементов ± 1 и удовлетворяющую следующему условию:*

$$H H^T = kE,$$

где H^T – транспонированная матрица H , а E – единичная матрица того же порядка.

Определение 6. *Матрица Сильвестра–Адамара порядка H_{2^n} – это квадратная матрица размера $2^n \times 2^n$, элементы которой равны ± 1 и которая строится по следующим рекуррентным соотношениям:*

$$H_1 = \begin{pmatrix} 1 \end{pmatrix}, \quad H_{2^n} = \begin{pmatrix} H_{2^{n-1}} & H_{2^{n-1}} \\ H_{2^{n-1}} & -H_{2^{n-1}} \end{pmatrix}.$$

Нетрудно убедиться в том, что матрица Сильвестра-Адамара эквивалентна матрице:

$$(h_{ij} : i, j \in \mathbb{Z}_2^n),$$

где $h_{ij} = (-1)^{(i,j)}$.

Введем *вектор таблицы истинности полярности* или *вектор характеристической последовательности функции* f : $F = ((-1)^{f(0^n)} \dots (-1)^{f(0^n)})$, а также вектор коэффициентов ПУА функции f : $\hat{F} = (\hat{f}(0^n) \dots \hat{f}(1^n))$. Из определения ПУА следует, что

$$\hat{F} = F \cdot H_{2^n}, \quad (7)$$

а из свойств матрицы Адамара получаем формулу обращения

$$F = 2^{-n} \hat{F} \cdot H_{2^n}.$$

2.4 Свойства булевых функций

Чтобы булева функция могла использоваться в криптографии, она должна обладать рядом специальных свойств, которые делают её устойчивой к взлому. К таким свойствам относятся: сбалансированность, нелинейность, алгебраическая степень и корреляционная иммунность. Каждое из них играет свою роль в защите от различных видов атак.

2.4.1 Сбалансированность

Булева функция f называется *уравновешенной* или *сбалансированной*, если $w(f) = 2^{n-1}$. Это означает, что функция принимает значение 1 и 0 на одинаковом числе наборов аргументов.

С точки зрения теории вероятностей, это эквивалентно утверждению, что при случайном выборе входного набора значение функции будет равно-

вероятно принимать 0 или 1:

$$\mathbb{P}(f(x) = 1) = \mathbb{P}(f(x) = 0).$$

Где

$$\mathbb{P}(f(x) = 1) = \frac{w(f)}{2^n}, \quad \mathbb{P}(f(x) = 0) = 1 - \frac{w(f)}{2^n}.$$

Сбалансированность обеспечивает равномерное распределение выходных значений функции, что крайне важно для обеспечения криптографической стойкости. Если функция несбалансирована, выходной бит становится предсказуемым с вероятностью, отличной от $1/2$, что может использоваться злоумышленником при анализе шифра.

Данное свойство особенно критично при проектировании S-блоков блочных шифров и функций фильтрации в потоковых шифрах. Например, в блочных шифрах несбалансированность хотя бы одной из компонентных булевых функций может привести к утечке информации через линейные аппроксимации или дифференциальные характеристики. В потоковых шифрах, в частности при использовании линейных сдвиговых регистров (LFSR), сбалансированные функции фильтрации препятствуют корреляционным атакам, которые эксплуатируют статистические зависимости между входами и выходом.

Таким образом, сбалансированность является необходимым условием криптографической применимости булевой функции. Отсутствие этого свойства делает невозможным достижение надёжной стойкости даже при наличии других полезных характеристик, таких как высокая нелинейность или корреляционная иммунность.

2.4.2 Нелинейность булевых функций

Пусть $f \in \mathcal{F}_n$, а $G \subseteq \mathcal{F}_n$ - некоторое подмножество булевых функций. Расстоянием $d(f, G)$ от функции f до класса функций G называется мини-

мальное расстояние от f до любой функции из G :

$$d(f, G) = \min_{g \in G} d(f, g),$$

где $d(f, g)$ - расстояние Хэмминга между функциями f и g .

Определение 7. *Нелинейностью функции [5] f называется расстояние от f до класса аффинных функций и обозначается N_f :*

$$N_f = d(f, \mathcal{A}(n)) = \min_{g \in \mathcal{A}} d(f, g).$$

По определению ПУА $\hat{f}(a) = 2^n - 2d(f, (a, x))$, отсюда:

$$d(f, (a, x)) = 2^{n-1} - \frac{1}{2}\hat{f}(a).$$

Тогда

$$d(f, (a, x) \oplus 1) = 2^n - d(f, (a, x)) = 2^{n-1} + \frac{1}{2}\hat{f}(a)$$

и

$$\min(d(f, (a, x)), d(f, (a, x) \oplus 1)) = 2^{n-1} - \frac{1}{2}|\hat{f}(a)|.$$

Если взять минимум по всем векторам a , получим формулу для N_f :

$$N_f = 2^{n-1} - \frac{1}{2} \max_{a \in \mathbb{Z}_2^n} |\hat{f}(a)|. \quad (8)$$

Для коэффициентов ПУА справедливо равенство Парсеваля. Для $f \in \mathcal{F}_n$:

$$\sum_a \hat{f}^2(a) = 2^{2n}.$$

Данное равенство позволяет оценить снизу максимум модуля коэффициентов ПУА: $\max |\hat{f}(a)| > 0$. Отсюда получаем тривиальную оценку для нелинейности: $N_f < 2^{n-1}$.

Определение 8. *Наилучшим аффинным приближением функции f называется аффинная функция, расстояние Хэмминга до которой минимально.*

Формула (8) позволяет не только вычислить нелинейность функции, но и найти наилучшее аффинное приближение: пусть $\max |\hat{f}(a)| = |\hat{f}(b)|$; тогда, если $\hat{f}(b) > 0$, то наилучшее аффинное приближение для функции f – функция (b, x) , если же $\hat{f}(b) < 0$ – функция $(b, x) \oplus 1$.

Пример 3. Пусть $f(x, y, z) = (10010111)$. Вычислим ПУА:

$\hat{f} = (-2, 2, 2, -2, 2, -2, -2, -6)$. Так как $\max |\hat{f}(a)| = -6 = \hat{f}(111)$, то наилучшее аффинное приближение функции f – это $x \oplus y \oplus z \oplus 1 = (10010110)$, и $d(f, (x \oplus y \oplus z \oplus 1)) = 1 = 2^2 - 6/2$.

Свойство нелинейности является одним из ключевых при проектировании криптографически стойких булевых функций, используемых в симметричных шифрах, особенно в поточных и блочных шифрах (например, в S-блоках). Чем выше нелинейность, тем сложнее выразить данную булеву функцию в виде простой линейной комбинации входных переменных. Высокая нелинейность необходима для защиты криптосистемы от следующих видов атак:

- **Линейный криптоанализ:** если булевы функции, используемые в шифре, имеют низкую нелинейность, то криптоаналитик может с высокой вероятностью аппроксимировать поведение шифра с помощью линейных уравнений, что позволяет восстанавливать ключи или открытые данные.
- **Дифференциальный криптоанализ:** хотя нелинейность напрямую не определяет устойчивость к этой атаке, высоконелинейные функции, как правило, затрудняют построение эффективных дифференциальных характеристик.

Таким образом, высокое значение нелинейности повышает криптографическую стойкость булевой функции и всей системы в целом, препятствуя

применению широкого класса аналитических атак. Это делает выбор булевых функций с высокой нелинейностью важным этапом при проектировании современных шифров.

2.4.3 Алгебраическая степень

Определение 9. *Степенью монома называется количество сомножителей в нём. Наибольшая степень монома в АНФ Булевой функции f называется степенью (алгебраической степенью) функции f ($\deg f$).*

Таким образом, функция f из примера 1 имеет степень 2.

Следствие 2. *Пусть $f \in \mathcal{F}_n$. Тогда $\deg f = n \Leftrightarrow w(f)$ нечётен.*

Доказательство. Так как $\deg f = n$, то $g(1^n) = 1$, где 1^n - единичный вектор длины n . По формуле (5) имеем:

$$g(1^n) = \bigoplus_{x \in \mathbb{Z}_2^n} f(x) = 1.$$

Это равенство возможно только в случае нечётного значения веса функции f , то есть при нечётном числе единичных значений $f(x)$ на всём пространстве. В противном случае сумма бы равнялась нулю. $\square$

Так как Булевых функций с нечетным весом ровно половина, то эта половина также будет иметь максимальную возможную степень.

Утверждение 4. *(о связи веса и степени функции).*

Пусть $f \in \mathcal{F}_n$, $\deg f = d \geq 1$. Тогда $2^{n-d} \leq w(f) \leq 2^n - 2^{n-d}$.

Доказательство. Разложим f по формуле (1) по $n-d$ переменным, не входящим в моном наибольшей степени. Пусть $f_1, \dots, f_{2^{n-d}}$ - коэффициенты этого разложения. Т.к. каждый коэффициент зависит как минимум от d переменных, то справедлива следующая оценка:

$$1 \leq w(f_i) \leq 2^d - 1 \quad \forall i \in [1, 2^{n-d}]$$

По утверждению (2):

$$w(f) = \sum_{i=1}^{2^{n-d}} w(f_i).$$

Отсюда получаем нужную оценку. □

Для дальнейшего анализа алгебраических свойств булевых функций важно ввести понятие аннигиляторов.

Определение 10. Аннигилятором булевой функции $f \in \mathcal{F}_n$ называется функция $g \in \mathcal{F}_n$, такая что:

$$f(x) \cdot g(x) = 0 \quad \forall x \in \mathbb{Z}_2^n$$

Если существует такой аннигилятор низкой алгебраической степени, то это может использоваться в алгебраических атаках на криптографическую систему, в которой используется функция f .

Алгебраическая степень булевой функции играет важную роль в оценке её устойчивости к различным видам атак, особенно тем, которые используют алгебраическое описание шифра. Однако сама по себе высокая степень не гарантирует защиту от алгебраических атак, поскольку даже функции высокой степени могут допускать существование низкостепенных аннигиляторов, что делает их уязвимыми.

В этом контексте естественным продолжением темы степени является понятие алгебраической иммунности, которое более точно характеризует устойчивость функции к алгебраическим атакам. Алгебраическая иммунность учитывает не только степень самой функции, но и существование любых функций низкой степени, которые её аннулируют. Поэтому дальнейшее исследование этой характеристики является важным этапом анализа криптографических свойств булевых функций и будет рассмотрено в следующих работах.

2.4.4 Корреляционная иммунность

Потоковые шифры подвержены корреляционным атакам, суть которых заключается в установлении зависимости между выходами функции и отдельными входными битами (или их линейными комбинациями) [6]. Для защиты от таких атак вводится понятие корреляционной иммунности булевой функции, которое отражает её устойчивость к подобным статистическим анализам. Чем выше корреляционная иммунность булевой функции, тем сложнее спрогнозировать влияние входных данных на выходные даже при частичном знании входного вектора.

Рассмотрим три определения корреляционной иммунности.

Пусть X, Z – случайные величины со значениями в U, V соответственно. X, Z – статистически независимы, если

$$\forall u \in U, \forall v \in V : \mathbb{P}\{X = u, Z = v\} = \mathbb{P}\{X = u\} \cdot \mathbb{P}\{Z = v\}.$$

Пусть $f \in \mathcal{F}_n$; $X^{(1)}, \dots, X^{(n)}$ – статистически независимые равномерно распределённые случайные величины со значениями в $\mathbb{Z}_2$ ($\mathbb{P}\{X^{(i)} = 0\} = \mathbb{P}\{X^{(i)} = 1\} = 1/2 \quad \forall i \in [1, n]$); $Z = f(X^{(1)}, \dots, X^{(n)})$ – случайная величина, тогда её распределение – $\mathbb{P}\{Z = 1\} = w(f)/2^n$, $\mathbb{P}\{Z = 0\} = 1 - w(f)/2^n$.

Определение 11. Если случайные величины $X^{(i_1, \dots, i_m)} = X^{(1)}, \dots, X^{(m)}$ ($0 < m \leq n$) и $Z = f(X^{(1)}, \dots, X^{(n)})$ ($f \in \mathcal{F}_n$) статистически независимы, тогда функция f статистически не зависит от подмножества своих переменных $\{x_{i_1}, \dots, x_{i_m}\}$.

Определение 12. (статистическое) Функция $f \in \mathcal{F}_n$ называется корреляционно-иммунной порядка m , $0 < m \leq n$, если она статистически не зависит от любого подмножества своих аргументов мощности m , или если $\forall (i_1, \dots, i_m), \quad 1 \leq i_1 < i_2 < \dots < i_m \leq n$ случайные величины $X^{(i_1, \dots, i_m)}$ и $Z = f(X^{(1)}, \dots, X^{(n)})$ – статистически независимы.

Статистическое определение независимости можно выразить через термины теории информации. Взаимная информация случайных величин X и Z выражается следующим образом:

$$I(X, Z) = H(X) - H(X|Z),$$

где $H(X)$ и $H(X|Z)$ - безусловные и условные энтропии случайной величины X соответственно. Таким образом, статистическую независимость случайных величин можно отождествить с их нулевой взаимной информацией, так как в таком случае значение случайной величины Z не добавляет информации к случайной величине X .

Определение 13. (теоретико-информационное) Функция $f \in \mathcal{F}_n$ называется корреляционно-иммунной порядка t , $0 < t \leq n$, если

$$\forall(i_1, \dots, i_m), 1 \leq i_1 < i_2 < \dots < i_m \leq n, I(X^{(i_1, \dots, i_m)}, f(X^{(1)}, \dots, X^{(n)})) = 0.$$

Подфункцией булевой функции f будем называть функцию, полученную в результате подстановки констант вместо некоторых переменных функции f .

Определение 14. (комбинаторное) Функция $f \in \mathcal{F}_n$ называется корреляционно-иммунной порядка t , если любая её подфункция от $n - t$ переменных имеет распределение равное распределению функции f , т.е. если бы выполнялось равенство $\mathbb{P}\{g = 1\} = \mathbb{P}\{f = 1\}$ или $w(g) = w(f)/2^m$.

Докажем равносильность определений (12) и (14). Заметим, что

$$\mathbb{P}\{X^{(i_1, \dots, i_m)} = a_1 \dots a_m\} = \frac{1}{2^m};$$

$$\mathbb{P}\{f = 1\} = \frac{w(f)}{2^n};$$

$$\mathbb{P}\{X^{(i_1, \dots, i_m)} = a_1 \dots a_m, f = 1\} = \frac{w(g)}{2^n},$$

где функция g получена подстановкой констант $a_1, \dots, a_m$ в функцию f вместо переменных $x_{i_1}, \dots, x_{i_m}$ соответственно. По (12) последнее равенство должно равняться произведению первых двух, в таком случае получаем:

$$\frac{w(f)}{2^n \cdot 2^m} = \frac{w(g)}{2^n}.$$

Следовательно $w(g) = w(f)/2^m$. Функция f уравновешена и является корреляционно-иммунной порядка $m \Leftrightarrow$ все ее подфункции от $n - m$ переменных уравновешены.

Действительно: $w(g) = w(f)/2^m = 2^{n-1}/2^m = 2^{n-m-1}$.

Определение 15. Уравновешенная корреляционно-иммунная порядка m функция называется m -устойчивой.

Утверждение 5. Функция $f \in \mathcal{F}_n$ – корреляционно-иммунная порядка $m \Leftrightarrow \hat{f}(a) = 0 \quad \forall a \in \mathbb{Z}_2^n : 1 \leq w(a) \leq m$ и является m -устойчивой $\Leftrightarrow \hat{f}(a) = 0 \quad \forall a \in \mathbb{Z}_2^n : 0 \leq w(a) \leq m$.

Свойство корреляционной иммунности булевой функции играет важную роль при проектировании криптографических примитивов, особенно в поточных шифрах, где булевы функции используются в качестве фильтрующих или комбинирующих функций. Корреляционная иммунность описывает устойчивость функции к корреляционным атакам, основанным на выявлении статистической зависимости между входными битами и выходным значением функции. Чем выше значение порядка, тем выше уровень корреляционной иммунности. Таким образом, высокая корреляционная иммунность уменьшает утечку информации о внутренних переменных генератора и значительно затрудняет проведение статистического анализа.

2.5 Производная булевой функции

Определим частную производную булевой функции $f \in \mathcal{F}_n$ по i -ой переменной как:

$$\left. \frac{\partial f}{\partial x_i} \right|_x = f(x_1, \dots, x_i, \dots, x_n) \oplus f(x_1, \dots, x_i \oplus 1, \dots, x_n) \quad (9)$$

Производная первого порядка показывает зависимость функции от своего i -ого аргумента. То есть $\partial f / \partial x_i = 1$, если при изменении значения x_i с неизменными аргументами $x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_n$ функция f меняет своё значение.

Рассмотрим и докажем свойства частной производной. Для удобства введем обозначение $x^{(i)} = (x_1, \dots, x_i \oplus 1, \dots, x_n)$.

$$1) \quad \frac{\partial(const)}{\partial x} = 0;$$

$$\frac{\partial(const)}{\partial x} = const \oplus const = 0;$$

$$2) \quad \frac{\partial \neg f}{\partial x} = \frac{\partial f}{\partial x};$$

$$\frac{\partial \neg f}{\partial x} = (f(x) \oplus 1) \oplus (f(x^{(i)}) \oplus 1) = f(x) \oplus f(x^{(i)}) = \frac{\partial f}{\partial x};$$

$$3) \quad \frac{\partial(f \oplus g)}{\partial x} = \frac{\partial f}{\partial x} \oplus \frac{\partial g}{\partial x};$$

$$\frac{\partial(f \oplus g)}{\partial x} = (f(x) \oplus g(x)) \oplus (f(x^{(i)}) \oplus g(x^{(i)})) = \frac{\partial f}{\partial x} \oplus \frac{\partial g}{\partial x};$$

$$4) \quad \frac{\partial}{\partial x_i} \left(\frac{\partial f}{\partial x_i} \right) = 0;$$

$$\begin{aligned} \frac{\partial}{\partial x_i} \left(\frac{\partial f}{\partial x_i} \right) &= \frac{\partial}{\partial x_i} (f(x) \oplus f(x^{(i)})) = \frac{\partial}{\partial x_i} (f(x)) \oplus \frac{\partial}{\partial x_i} (f(x^{(i)})) = \\ &= f(x) \oplus f(x^{(i)}) \oplus f(x^{(i)}) \oplus f(x) = 0; \end{aligned}$$

$$5) \quad \frac{\partial}{\partial x_i} \left(\frac{\partial f}{\partial x_j} \right) = \frac{\partial}{\partial x_j} \left(\frac{\partial f}{\partial x_i} \right);$$

$$\begin{aligned} \frac{\partial}{\partial x_i} \left(\frac{\partial f}{\partial x_j} \right) &= \frac{\partial}{\partial x_i} (f(x) \oplus f(x^{(j)})) = \frac{\partial}{\partial x_i} (f(x)) \oplus \frac{\partial}{\partial x_i} (f(x^{(j)})) = \\ &= f(x) \oplus f(x^{(i)}) \oplus f(x^{(j)}) \oplus f(x^{(ij)}) = \frac{\partial}{\partial x_j} (f(x)) \oplus \frac{\partial}{\partial x_j} (f(x^{(i)})) = \\ &= \frac{\partial}{\partial x_j} (f(x) \oplus f(x^{(i)})) = \frac{\partial}{\partial x_j} \left(\frac{\partial f}{\partial x_i} \right); \end{aligned}$$

$$6) \quad \frac{\partial(f \cdot g)}{\partial x} = f \frac{\partial g}{\partial x} \oplus g \frac{\partial f}{\partial x} \oplus \frac{\partial f}{\partial x} \frac{\partial g}{\partial x};$$

$$\frac{\partial(f \cdot g)}{\partial x_i} = f(x) \cdot g(x) \oplus f(x^{(i)}) \cdot g(x^{(i)}) = f(x) \cdot g(x) \oplus \left(\frac{\partial f}{\partial x_i} \oplus f(x) \right) \cdot g(x).$$

$$\left(\frac{\partial g}{\partial x_i} \oplus g(x)\right) = f(x) \cdot g(x) \oplus \left(\frac{\partial f}{\partial x_i} \frac{\partial g}{\partial x_i} \oplus \frac{\partial f}{\partial x_i} g \oplus f \frac{\partial g}{\partial x_i} \oplus f(x) \cdot g(x)\right) = f \frac{\partial g}{\partial x} \oplus g \frac{\partial f}{\partial x} \oplus \frac{\partial f}{\partial x} \frac{\partial g}{\partial x};$$

$$7) \frac{\partial(f \vee g)}{\partial x} = \neg f \frac{\partial g}{\partial x} \oplus \neg g \frac{\partial f}{\partial x} \oplus \frac{\partial f}{\partial x} \frac{\partial g}{\partial x};$$

Воспользуемся формулой: $f \vee g = f \oplus g \oplus f \cdot g$

$$\begin{aligned} \frac{\partial(f \vee g)}{\partial x} &= \frac{\partial(f \oplus g \oplus f \cdot g)}{\partial x} = \frac{\partial f}{\partial x} \oplus \frac{\partial g}{\partial x} \oplus \frac{\partial(f \cdot g)}{\partial x} = \frac{\partial f}{\partial x} \oplus \frac{\partial g}{\partial x} \oplus f \frac{\partial g}{\partial x} \oplus g \frac{\partial f}{\partial x} \oplus \frac{\partial f}{\partial x} \frac{\partial g}{\partial x} = \\ &= (f \oplus 1) \frac{\partial g}{\partial x} \oplus (g \oplus 1) \frac{\partial f}{\partial x} \oplus \frac{\partial f}{\partial x} \frac{\partial g}{\partial x} = \neg f \frac{\partial g}{\partial x} \oplus \neg g \frac{\partial f}{\partial x} \oplus \frac{\partial f}{\partial x} \frac{\partial g}{\partial x}. \end{aligned}$$

Определение 16. Говорят, что функция $f(x_1, \dots, x_n)$ линейно зависит от переменной x_i , если f представима в виде:

$$f(x_1, \dots, x_n) = g(x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_n) \oplus x_i,$$

Или если

$$f(x_1, \dots, x_{i-1}, 0, x_{i+1}, \dots, x_n) \neq f(x_1, \dots, x_{i-1}, 1, x_{i+1}, \dots, x_n).$$

С учетом определения частной производной (9) понятие линейности функции от переменной x_i можно записать как:

$$\frac{\partial f}{\partial x_i} = 1$$

Определение 17. Производная функции f по направлению a называется функция $f'_a = f(x) \oplus f(x \oplus a)$.

Пример 4. (Пример производной по направлению для функции от 2 переменных)

x	$f(x)$	f'_{00}	f'_{01}	f'_{10}	f'_{11}
00	0	0	1	1	0
01	1	0	1	1	0
10	1	0	1	1	0
11	0	0	1	1	0

Рассмотрим и докажем простейшие свойства производной по направлению.

$$1) f'_0(x) \equiv 0;$$

$$f'_0(x) = f(x) \oplus f(x \oplus 0) \equiv 0$$

$$2) f'_a(x) = f'_a(x \oplus a);$$

$$f'_a(x \oplus a) = f(x \oplus a) \oplus f(x \oplus a \oplus a) = f'_a(x);$$

$$3) (f \oplus g)'_a = f'_a \oplus g'_a;$$

$$(f \oplus g)'_a = f(x) \oplus g(x) \oplus f(x \oplus a) \oplus g(x \oplus a) = f'_a \oplus g'_a;$$

$$4) f'_{a \oplus b}(x) = f'_a(x) \oplus f'_b(x \oplus a);$$

$$f'_{a \oplus b}(x) = f(x)f(x \oplus a \oplus b) = f(x) \oplus f(x \oplus a) \oplus f(x \oplus a) \oplus f(x \oplus a \oplus b) = f'_a(x) \oplus f'_b(x \oplus a).$$

ВЫВОДЫ

В данном разделе рассмотрены ключевые свойства булевых функций, имеющих значение для криптографического применения. Приведено определение производной булевой функции по направлению, проанализированы и доказаны её основные характеристики. Особое внимание уделено важнейшим криптографическим свойствам булевых функций, таким как балансированность, нелинейность, алгебраическая степень и корреляционная иммунность, поскольку они напрямую влияют на устойчивость шифров к различным видам атак. Рассмотренные преобразования Мёбиуса и Уолша-Адамара, а также их быстрые алгоритмы, позволяют эффективно вычислять спектральные характеристики булевых функций.

В дальнейшем планируется рассмотреть свойство алгебраической иммунности, а также применение дифференциальных свойств булевых функций в криптографическом анализе.

3 Практическая часть

Для реализации программы по анализу булевых функций выбран язык *Python*. Этот выбор обусловлен рядом причин:

1) высокий уровень абстракции. *Python* позволяет сосредоточиться на логике алгоритмов, минимизируя необходимость работы с низкоуровневыми деталями, такими как управление памятью или типизация переменных;

2) читаемость и удобство поддержки кода. Благодаря лаконичному синтаксису и широкому распространению в научной среде, *Python* идеально подходит для реализации прототипов криптографических алгоритмов и последующего анализа их свойств;

3) богатая экосистема библиотек. Наличие таких библиотек, как *NumPy*, *SymPy*, *Matplotlib*, *Seaborn* и других, существенно облегчает реализацию математических и визуальных компонентов программы.

Однако, несмотря на удобство и выразительность, *Python* является интерпретируемым языком, что может отрицательно сказываться на производительности при выполнении ресурсоёмких вычислений. Для устранения этого недостатка в разработке использовалась библиотека *Numba*, позволяющая применять JIT-компиляцию. В частности, декоратор *@njit* (*No-Python Just-In-Time compilation*) позволил компилировать функции в машинный код во время выполнения, что привело к значительному увеличению скорости выполнения критичных участков программы.

Также в рамках разработки реализована система адаптивного выбора алгоритма в зависимости от количества входных переменных булевой функции. При малом числе переменных использовалась простая и быстрая реализация, тогда как при превышении определённого порогового значения программа автоматически переключалась на более универсальный, но также более оптимизированный алгоритм.

Особое внимание уделено применению эффективных математических

преобразований [7], таких как:

- быстрое преобразование Мёбиуса, которое используется для получения спектра функции по её значению на подмножествах переменных;
- быстрое преобразование Уолша-Адамара, позволяющее вычислять спектр Уолша булевой функции и тем самым анализировать такие характеристики, как корреляционная иммунность и нелинейность;
- другие вспомогательные преобразования.

Применение данных оптимизаций позволило обеспечить высокую производительность программы при анализе функций с большим числом переменных и существенно сократить время выполнения вычислений.

3.1 Программная реализация

Программная реализация начинается с построения класса *BoolFunc* (приложение А), предназначенного для хранения и анализа булевых функций в представлении таблицы истинности (вектора значений). Конструктор класса осуществляет базовую валидацию входных данных: строка значений должна состоять только из символов '0' и '1', а её длина – являться степенью двойки, что гарантирует возможность представления функции от n переменных. Далее строка преобразуется в массив целых чисел, упаковывается в байты, и инициализируются вспомогательные структуры, используемые при анализе функции:

- *self.size* – размер вектора значений;
- *self.n* – количество переменных функции;
- *self.tv* – вектор значений функции;
- *self.tv_packed* – упакованный в массив из байтов вектор значений;
- *self._sv* – знаковый вектор;
- *self._w* – вес функции;
- *self._anf* – АНФ представление функции;
- *self._walsh_spec* – спектр функции.

Помимо основного конструктора, реализующего инициализацию через массив значений, класс содержит несколько дополнительных классовых методов (*@classmethod*), обеспечивающих гибкую инициализацию из различных форматов данных:

- метод *from_array(cls, arr: np.ndarray)* позволяет создать экземпляр *BoolFunc* на основе массива *numpy*, содержащего значения таблицы истинности. Массив должен состоять исключительно из нулей и единиц (*dtype=np.uint8*), а его длина — являться степенью двойки, соответствующей числу переменных;

- метод *from_packed(cls, packed: np.ndarray, n: int = None)* служит для создания объекта на основе упакованного массива байтов, где каждый байт хранит 8 битов булевой функции. Если количество переменных меньше трех, то аргумент *n* позволяет явно указать число переменных; если он не задан, длина массива определяется автоматически;

- метод *from_bytes(cls, byte_data: bytes, n: int = None)* инициализирует объект *BoolFunc* на основе последовательности байтов. Это удобно при чтении функции из бинарных файлов или при передаче по сети. В отличие от *from_packed*, метод работает напрямую с байтовыми объектами *bytes*;

- метод *random(cls, n: int)* создаёт случайную булеву функцию от *n* переменных. Для генерации используется встроенный генератор случайных чисел с равномерным распределением *np.random.default_rng()*. Полученная функция представляет собой случайный вектор длиной 2^n , заполненный нулями и единицами. Временная сложность — $O(2^n)$;

- метод *load_from_bin(cls, filename: str)* загружает объект *BoolFunc* из бинарного файла, сохранённого в специальном формате. Файл должен содержать: один байт, обозначающий число переменных *n*; далее следуют байты, представляющие упакованную таблицу истинности. Этот метод предполагает, что файл создан с использованием метода *save_to_bin*. Аргументы функции *save_to_bin* включают массив *np.ndarray*, содержащий данные, подлежа-

щие сохранению, и строку *filename*, указывающую путь к целевому файлу. Следует отметить, что использование двоичного формата существенно повышает эффективность работы с большими массивами данных по сравнению с текстовыми форматами особенно в контексте численно-ориентированных вычислений.

После загрузки и предварительной обработки данных осуществляется вычисление различных характеристик булевой функции. В данной реализации применён подход ленивой инициализации (*lazy evaluation*) – значения вычисляются по требованию и сохраняются в приватные атрибуты экземпляра для последующего быстрого доступа:

- метод *sv(self)* представляет собой преобразование таблицы истинности в знаковый вектор (*sign vector*) : значения 0 заменяются на +1, а 1 на –1. Такое представление используется для вычисления спектра. Свойство кэширует результат в переменной *_sv*. Временная сложность – $O(n)$;

- свойства *popcount_table8* и *popcount_table16* предназначены для ускорения подсчёта количества установленных битов в векторе. Для этого используется предвычисленная таблица, загружаемая из файлов *popcount8.npz* и *popcount16.npz*. Эти таблицы позволяют заменить дорогостоящую по времени операцию подсчёта единиц на операцию индексирования массива. Подобная оптимизация актуальна при анализе булевых функций высокой размерности;

- свойство *w(self)* – вычисление и кэширование веса функции. При небольших значениях *n* (размерность функции) используется стандартный подсчёт суммы массива *NumPy*. При больших *n*, таблица упаковывается в байты (*tv_packed*), с применением таблицы *popcount_table8*, позволяющей значительно ускорить вычисления. Временная сложность – $O(2^n)$;

- свойство *anf(self)* – вычисляет АНФ булевой функции с помощью быстрого преобразования Мебиуса – функции *fnt(g, n)*, основанной на формуле (5). Суть быстрого преобразования Мебиуса заключается в следующем:

массив g , содержащий значения функции на всех подмножествах множества из n элементов, индексируется числами от 0 до $2^n - 1$, которые можно интерпретировать как битовые маски длины n . Алгоритм последовательно перебирает разряды маски i от 0 до $n-1$ и обновляет значения в массиве по следующему правилу: если два индекса отличаются только в i -м бите, то значение по индексу с установленным битом обновляется по формуле:

$$g[a] \oplus = g[a \oplus (1 \ll i)].$$

Таким образом, по завершении работы алгоритма в массиве g хранятся значения преобразованной функции, где каждый элемент представляет собой накопленный *XOR* значений функции по всем маскам, являющимся подмасками соответствующего индекса.

Временная сложность алгоритма оценивается следующим образом. Внешний цикл выполняется n раз (по разрядам). Во вложенном цикле *block_start* проходит по всем 2^n элементам с шагом $2^i + 1$, т.е. всего выполняется 2^{n-i-1} итераций. В каждой итерации этого цикла внутренний цикл по j выполняется 2^i раз. Таким образом, общее число операций на каждом шаге i составляет: $2^{n-i-1} \cdot 2^i = 2^{n-1}$. Так как таких шагов n , общая сложность алгоритма — $O(n \cdot 2^n)$;

- свойство *walsh_spec* предназначено для ленивого вычисления спектра булевой функции с помощью быстрого преобразования Уолша–Адамара, реализованного в функции *fwht(arr)*, основанной на формуле (7). Результат однократно вычисляется при первом обращении и сохраняется в поле *self._walsh_spec* для повторного использования. Функция *fwht(arr)* имитирует умножение знакового вектора на матрицу Адамара–Сильвестра, используя алгоритм «бабочка». Идея алгоритма заключается в поэтапной обработке входного массива блоками переменной длины, являющейся степенью двойки. Временная сложность алгоритма оценивается следующим

образом. Внешний цикл по h выполняется $\log_2 n$ раз, так как h удваивается от 1 до n . На каждом уровне весь массив длины n делится на $n/(2h)$ блоков. В каждом блоке обрабатывается h пар, то есть общее количество операций – $n/(2h) \cdot h = n/2$. Таким образом, на каждом уровне выполняется $O(n)$ операций, а уровней всего $\log_2 n$, следовательно временная сложность – $O(n \log_2 n)$;

- свойство *is_balanced(self)* реализует проверку свойства сбалансированности функции. При кэшированном значении веса временная сложность – $O(1)$, при неинициализированном весе – $O(2^n)$;

- метод *hamming_distance(self, other: np.ndarray)* вычисляет расстояние Хэмминга между объектами класса *BoolFunc* с помощью формулы (6). Временная сложность – $O(n)$;

- метод *nonlinearity(self)* вычисляет нелинейность булевой функции по формуле (8). Абсолютное значение максимального коэффициента Уолша определяется с помощью одной из двух функций: *max_abs_par(self.walsh_spec)* или *max_abs_seq(self.walsh_spec)*, выбор между которыми зависит от количества переменных функции. Функция *max_abs_par* использует параллельные вычисления, и её временная сложность оценивается как $O(n/p + p)$, где n – размер спектра Уолша, а p – количество потоков. В свою очередь, *max_abs_seq* выполняется последовательно и имеет временную сложность $O(n)$;

- метод *best_affine_approximation(self)*, также реализованный по формуле (8), реализует поиск лучшего аффинного приближения. Поскольку преобразование Мёбиуса является наиболее ресурсоёмкой частью метода *best_affine_approximation*, его асимптотика доминирует над остальными компонентами. Следовательно, общая временная сложность метода составляет $O(n \cdot 2^n)$;

- метод *algebraic_degree(self)* реализует вычисление алгебраической степени булевой функции по её представлению АНФ. В зависимости от до-

ступной информации (веса функции и числа переменных), метод выбирает одну из нескольких стратегий вычисления. Если известен вес функции и он нечётный, то, согласно Следствию (2), функция имеет максимальную степень n . При небольшом числе переменных ($n \leq 12$) используется полное переборное вычисление с ограничением по верхней оценке степени, полученной из утверждения (4). Для $n > 12$ применяется оптимизированный поиск среди потенциальных степеней, начиная с максимальных, с использованием заранее сгенерированных индексов мономов нужной степени. Если вес неизвестен и $n \geq 12$, применяется вспомогательная функция *algebraic_degree_anf*, анализирующая АНФ напрямую. Во всех случаях, если оптимизации не применимы, используется прямой перебор всех ненулевых мономов с определением наибольшей степени по числу установленных битов в соответствующих индексах. Худший случай для метода *algebraic_degree(self)* реализуется при отсутствии информации о весе функции (*self.w is None*) и при большом числе переменных n . В этом случае используется полный перебор коэффициентов АНФ, где для каждого ненулевого монома производится подсчёт его степени. Таким образом, временная сложность метода в худшем случае составляет $O(n \cdot 2^n)$;

- метод *is_correlation_immune(self, order: int)* реализует проверку корреляционной иммунности булевой функции порядка m на основе спектрального критерия, изложенного в Утверждении (5). В зависимости от количества переменных выбирается метод генерации масок с Хэммингом весом, не превышающим заданный порядок. Далее производится проверка того, что значения спектра Уолша на всех сгенерированных масках равны нулю. Временная сложность генераторов масок – $O(2^n)$;

- метод *boolean_derivative(self, a: int)* реализует вычисление производной по направлению a по Определению (17). Временная сложность – $O(2^n)$;

- метод *find_fictive_vars(self)* предназначен для выявления фиктивных переменных в булевой функции, представленной в АНФ. Алгоритм пере-

бирает ненулевые коэффициенты и с помощью побитовой операции ИЛИ накапливает маску задействованных переменных. Фиктивные переменные определяются как те, чьи биты остаются нулевыми в итоговой маске. Результат возвращается в виде списка номеров фиктивных переменных. Временная сложность – $O(2^n)$.

Для демонстрации работы был разработан пользовательский интерфейс, изображённый на рисунке 3. Вывод результата приведён на рисунке 4.

```
if __name__ == "__main__":
    f = BoolFunc("01111010")
    print(f'Вектор значений: {f.tv}')
    print(f'Знаковый вектор: {f.sv}')
    print(f'Вес: {f.w}')
    print(f'Вектор АНФ: {f.anf}')
    print(f'АНФ: {f.visualize_anf()}')
    print(f'Спектр: {f.walsh_spec}')
    print(f'Сбалансированность: {f.is_balanced}')
    print(f'Нелинейность: {f.nonlinearity}')
    print(f'Лучшее аффинное приближение: {f.best_affine_approximation}')
    g = BoolFunc.from_array(f.best_affine_approximation)
    print(f'АНФ аффинного приближения: {g.visualize_anf()}')
    print(f'Алгебраическая степень: {f.algebraic_degree}')
    print(f'Корреляционная иммунность порядка 2: {f.is_correlation_immune(2)}')
    print(f'Фиктивные переменные: {f.find_fictive_vars}')
    print("Производные по всем направлениям:")
    print("-----")
    for i in range(8):
        print(f"Производная по направлению {bin(i)}")
        d = BoolFunc.from_array(f.boolean_derivative(i))
        print(d.visualize_anf())
    print("-----")
```

Рисунок 3 – Пользовательский интерфейс

```

/home/fedos/PycharmProjects/BoolFuncPy/.venv/bin/python /home/fedos/PycharmProjects/BoolFuncPy/main.py
Вектор значений: [0 1 1 1 1 0 1 0]
Знаковый вектор: [ 1 -1 -1 -1 -1  1 -1  1]
Вес: 5
Вектор АНФ: [0 1 1 1 1 0 1 1]
АНФ:  $x_1 \oplus x_2 \oplus x_1x_2 \oplus x_3 \oplus x_2x_3 \oplus x_1x_2x_3$ 
Спектр: [-2 -2  2  2 -2  6  2  2]
Сбалансированность: False
Нелинейность: 1
Лучшее аффинное приближение: [0 1 0 1 1 0 1 0]
АНФ аффинного приближения:  $x_1 \oplus x_3$ 
Алгебраическая степень: 3
Корреляционная иммунность порядка 2: False
Фиктивные переменные: None
Производные по всем направлениям:
-----
Производная по направлению 0b0
0
Производная по направлению 0b1
 $1 \oplus x_2 \oplus x_2x_3$ 
Производная по направлению 0b10
 $1 \oplus x_1 \oplus x_3 \oplus x_1x_3$ 
Производная по направлению 0b11
 $1 \oplus x_1 \oplus x_2 \oplus x_1x_3 \oplus x_2x_3$ 
Производная по направлению 0b100
 $1 \oplus x_2 \oplus x_1x_2$ 
Производная по направлению 0b101
 $x_2 \oplus x_1x_2 \oplus x_2x_3$ 
Производная по направлению 0b110
 $1 \oplus x_2 \oplus x_1x_2 \oplus x_3 \oplus x_1x_3$ 
Производная по направлению 0b111
 $x_2 \oplus x_1x_2 \oplus x_1x_3 \oplus x_2x_3$ 
-----
Process finished with exit code 0

```

Рисунок 4 – Результат работы программы

ВЫВОДЫ

В рамках данного раздела разработана программная библиотека на языке *Python*, обеспечивающая эффективную работу с булевыми функциями различных переменных. В реализации применены современные подходы к ускорению вычислений: использование библиотеки *NumPy* для работы с массивами и *pumba* для *JIT*-компиляции ресурсоёмких операций. Построенный программный модуль обеспечивает высокую вычислительную эффективность. Данный функционал создаёт надёжную основу для анализа булевых функций в контексте криптографических приложений и теоретических исследований.

ЗАКЛЮЧЕНИЕ

В курсовой работе проведено исследование свойств булевых функций, применяемых в криптографии, рассмотрено понятие производной булевой функции и ее свойства. Также разработан инструмент, предназначенный для эффективного анализа булевых функций.

Рассмотрены ключевые характеристики булевых функций, определяющие их пригодность для криптографического использования: сбалансированность, нелинейность, алгебраическая степень, корреляционная и дифференциальная иммунность. Проанализированы преобразования Мёбиуса и Уолша-Адамара, включая их быстрые алгоритмические реализации, что позволило упростить и ускорить вычисление спектральных характеристик. Произведен анализ производной булевой функции по направлению и её основных свойств, имеющих значение при анализе устойчивости к криптоаналитическим атакам.

Разработанный программный инструмент обеспечивает автоматизированный расчёт свойств булевых функций и их представление, что способствует дальнейшим исследованиям и выбору оптимальных функций для криптографических целей.

В качестве следующего этапа планируется сосредоточить внимание на изучении свойств алгебраической иммунности, а также провести более глубокий анализ устойчивости булевых функций к современным видам атак. Особый акцент предполагается сделать на подборе булевых функций, устойчивых к дифференциальному криптоанализу, что особенно важно при проектировании S-блоков блочных шифров и регистров сдвига с обратной связью в потоковых шифрах.

Полученные результаты формируют основу для дальнейших теоретических и практических исследований в области криптографического анализа булевых функций.

СПИСОК СОКРАЩЕННЫХ ОБОЗНАЧЕНИЙ

АНФ – Алгебраическая нормальная форма

ММИ – Метод математической индукции

ПУА – Преобразование Уолша-Адамара

СДНФ – Совершенная дизъюнктивная нормальная форма

СКНФ – Совершенная конъюнктивная нормальная форма

S-Box – *Substitution Box*, таблица замен

SV – *Sign Vector*, вектор значений

DES – *Data Encryption Standard*, стандарт шифрования данных

LFSR – *Linear Feedback Shift Register*, регистр сдвига с линейной обратной связью

NJIT – *No Python Just In Time Compilation*, отсутствие компиляции "на лету"

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Карле, К. Булевы функции для криптографии и теории кодирования [Текст] / К. Карле. – Кембридж: Кембриджский университетский пресс, 2021. – 588 с.
2. Алферов, А. П. Основы криптографии [Текст]: учебное пособие / А. П. Алферов, А. Ю. Зубов, А. С. Кузьмин, А. В. Черемушкин. – 2-е изд., испр. и доп. – М.: Гелиос АРВ, 2002. – 480 с.: ил.
3. Бабаш, А. В. Криптография [Текст]: учебное пособие / А. В. Бабаш, Г. П. Шанкин. М.: Г. П. СОЛОН-Р, 2002.
4. Панкратова, И. А. Булевы функции в криптографии [Текст]: учебное пособие / И. А. Панкратова. – Томск: Издательский дом Томского государственного университета, 2014. – 88 с.
5. Токарева Н. Н. Нелинейные булевы функции: бент-функции и их обобщения [Текст]. – Saarbrücken: LAP LAMBERT Academic Publishing, 2011.
6. Таранников Ю. В. О корреляционно-иммунных и устойчивых булевых функциях [Текст] // Математические вопросы кибернетики. – 2002. – Вып. 11. – С. 91–148.
7. Шнайер, Б. Прикладная криптография: протоколы, алгоритмы и исходные тексты на языке Си [Текст] пер. с англ. – М.: Триумф, 2002. – 816 с.

ПРИЛОЖЕНИЕ А

Программная реализация класса булевых функций

```
import numpy as np
from numba import njit, prange

class BoolFunc:
    def __init__(self, truth_vector: str):
        # Корректность содержимого введенной строки: строка должна содержать
        # только 0 и 1;
        if set(truth_vector) - {'0', '1'}:
            raise ValueError(f"Ошибка: вектор значений {truth_vector}
            содержит символы, отличные от '0' и '1'.")
        size = len(truth_vector) # 2^n
        # Корректность размера введенной строки: размер > 0 и размер = 2^n;
        if not (size > 0 and (size & (size - 1)) == 0):
            raise ValueError("Длина вектора значений должна быть степенью
            двойки!")

        self.size = size # Размер вектора значений
        self.n = self.size.bit_length() - 1 # log2(size)

        self.tv = np.frombuffer(truth_vector.encode(), dtype=np.uint8) -
        ord('0') # tv - truth vector, вектор значений
        '''Быстрое преобразование строки с бинарными данными ("0110...") в
        числовой массив для математических операций:
        - .encode() преобразует строку в байтовый объект тип( bytes),
        используя кодировку по умолчанию обычно( UTF-8);
        - np.frombuffer интерпретирует байты как массив чисел типа uint8;
        - (- ord('0')) - преобразует ASCIIкоды- цифр в соответствующие
        числа.'''

        # Упаковывает tv в байты (uint8), каждые 8 бит упакованы в один байт.
        self.tv_packed = np.packbits(self.tv, bitorder='big')

        # Отложенные вычисления, инициализация при необходимости
        self._sv = None # sign_vector - знаковый или полярный вектор.
        self._popcount_table8 = None
```

```

self._popcount_table16 = None
self._w = None # Бес.
self._anf = None
self._walsh_spec = None

@classmethod
def from_array(cls, arr: np.ndarray):
    if arr.dtype != np.uint8 or not np.isin(arr, [0, 1]).all():
        raise ValueError("Массив должен содержать только 0 и 1 (uint8).")
    size = arr.size
    if size == 0 or (size & (size - 1)) != 0:
        raise ValueError("Размер массива должен быть больше 0 и степенью
двойки.")

    obj = cls.__new__(cls) # Низкоуровневый способ создать объект
    класса, в обход __init__ методу.
    obj.size = size
    obj.n = size.bit_length() - 1
    obj.tv = arr
    obj.tv_packed = np.packbits(arr)
    obj._sv = obj._popcount_table8 = obj._popcount_table16 = obj._w =
obj._anf = obj._walsh_spec = None
    return obj

@classmethod
def from_packed(cls, packed: np.ndarray, n: int = None):
    bits = np.unpackbits(packed, bitorder='big')
    if n is not None:
        size = 1 << n
        if bits.size < size:
            raise ValueError(f"Недостаточно данных: требуется {size}
бит, получено {bits.size}")
        bits = bits[:size]
    return cls.from_array(bits)

@classmethod
def from_bytes(cls, byte_data: bytes, n: int = None):
    packed = np.frombuffer(byte_data, dtype=np.uint8)
    return cls.from_packed(packed, n)

```

```

@classmethod
def random(cls, n: int):
    size = 1 << n

    # Объект генератора псевдослучайных чисел, основанный на
    современном алгоритме PCG64
    rng = np.random.default_rng()
    tv = rng.integers(0, 2, size=size, dtype=np.uint8) # Случайный
    массив 0 и 1 размера size
    return cls.from_array(tv)

def save_to_bin(self, filename: str) -> None:
    """
    Сохраняет булеву функцию в бинарный файл:
    [1 байт nупакованные][ значения tv_packed]
    """
    with open(filename, 'wb') as f:
        f.write(bytes([self.n]))
        f.write(self.tv_packed.tobytes())

@classmethod
def load_from_bin(cls, filename: str) -> 'BoolFunc':
    with open(filename, 'rb') as f:
        n = int.from_bytes(f.read(1), 'big')
        packed = np.frombuffer(f.read(), dtype=np.uint8)
    return cls.from_packed(packed, n=n)

@property
def sv(self):
    """Вычисление знакового вектора: замена в векторе значений 0 -> 1, 1
    -> -1"""
    if self._sv is None:
        self._sv = 1 - 2 * self.tv.astype(np.int8)
    return self._sv

@property
def popcount_table8(self):
    """Загрузка popcount_table8 из файла"""
    if self._popcount_table8 is None:
        self._popcount_table8 = np.load("popcount8.npz")["popcount8"]

```

```

        return self._popcount_table8

@property
def popcount_table16(self):
    """Загрузка popcount_table16 из файла"""
    if self._popcount_table16 is None:
        self._popcount_table16 = np.load("popcount16.npz")["popcount16"]
    return self._popcount_table16

@property
def w(self):
    """Вычисление веса функции."""
    if self._w is None:
        if self.n >= 16:
            self._w = int(self.popcount_table8[self.tv_packed].sum())
        else:
            self._w = int(np.sum(self.tv))
    return self._w

@property
def anf(self):
    """Вычисление АНФ функции с помощью быстрого преобразование
Мёбиуса"""
    if self._anf is None:
        fmt([0, 1, 1, 0], 2)
        self._anf = self.mobius_transform(self.tv.copy(), self.n)
    return self._anf

@property
def walsh_spec(self):
    """Вычисление спектра с помощью быстрого преобразования
УолшаАдамара-"""
    if self._walsh_spec is None:
        self._walsh_spec = fwht(self.sv.astype(np.int64))
    return self._walsh_spec

@property
def is_balanced(self) -> bool:
    return self.w == 1 << (self.n - 1)

```

```

def hamming_distance(self, other: np.ndarray) -> int:
    """Расстояние Хэмминга"""
    if not isinstance(other, np.ndarray):
        raise TypeError("Ожидается NumPy массив-.")
    if other.shape != self.tv.shape:
        raise ValueError("Размерность другого вектора должна совпадать.")

    return int(np.sum(self.tv ^ other))

@staticmethod
def mobius_transform(f, n):
    return fmt(f, n)

@property
def nonlinearity(self):
    if self.n > 16:
        max_abs = max_abs_par(self.walsh_spec)[0][0]
    else:
        max_abs = max_abs_seq(self.walsh_spec)[0][0]
    return (1 << (self.n - 1)) - (abs(max_abs) >> 1)

@property
def best_affine_approximation(self):
    max_abs = max_abs_par(self.walsh_spec) if self.n >= 20 else
max_abs_seq(self.walsh_spec)
    min_dist = self.size
    best_appr = None
    for coeff, index in max_abs:
        a = int_to_bitarray(index, self.n)
        b_bit = 0 if coeff > 0 else 1
        anf = self.linear_function(a)
        candidate = self.mobius_transform(anf, self.n) ^ b_bit
        dist = self.hamming_distance(candidate)
        if dist < min_dist:
            min_dist = dist
            best_appr = candidate

    return best_appr

@staticmethod

```

```

def linear_function(a: np.ndarray):
    packed = linear_function_bitpacked_seq(a) if a.shape[0] < 20 else
linear_function_bitpacked_par(a)
    return np.unpackbits(packed, bitorder='little')[:1 << a.shape[0]]

@property
def algebraic_degree(self):
    if self.w is not None:
        if self.w % 2 != 0:
            return self.n
        if self.n <= 12:
            d_max = upper_deg(self.w, self.n)
            return max(i.bit_count() for i in range(1 << self.n) if
self.anf[i] and i.bit_count() <= d_max)
        elif self.n > 12:
            degree_candidates = get_degree_candidates(self.w, self.n)
            degree_index_sets = [generate_bitmask_indices_fast(self.n, d)
for d in degree_candidates]
            for i in range(len(degree_index_sets) - 1, -1, -1):
                for ind in degree_index_sets[i]:
                    if self.anf[ind]:
                        return degree_candidates[i]
        else:
            if self.n >= 12:
                return algebraic_degree_anf(self.anf)
            return max(i.bit_count() for i in range(self.size) if self.anf[i])

def is_correlation_immune(self, order: int) -> bool:
    """Проверяет, является ли функция корреляционноиммунной- заданного
порядка."""
    if order < 0:
        return True

    # Для n < 16 используем быстрый метод с popcount
    if self.n < 16:
        indices = self.generate_by_popcount(order)
        return np.all(self.walsh_spec[indices] == 0)

    # Для n >= 16 используем генератор combinations
    for a in self.generate_by_combinations(order):

```



```

        if self.walsh_spec[a] != 0:
            return False
    return True

def generate_by_popcount(self, m: int):
    """Генерация булевых векторов с весом <= m через popcount
    эффективно( для n ≤ 16)."""
    return fgbp(self.size, m, self.popcount_table16)

def generate_by_combinations(self, m: int):
    """Генерация булевых векторов с весом <= m через combinations
    эффективно( для n > 16)."""
    return generate_vectors_fast(self.n, m)

def boolean_derivative(self, a: int):
    if a >= self.size:
        raise ValueError(f"Некорректное направление: требуется
направление размером {self.n} бит.")
    if a == 0:
        return np.zeros(self.size, dtype=np.uint8)
    x_a = np.arange(self.size) ^ a
    return np.array([self.tv[i] ^ self.tv[x_a[i]] for i in
range(self.size)])

def visualize_anf(self):
    terms = []
    if not np.any(self.tv):
        return 0
    for i, coeff in enumerate(self.anf):
        if coeff == 0:
            continue
        if i == 0:
            terms.append("1")
            continue
        monomial = []
        for j in range(self.n):
            if (i >> j) & 1:
                monomial.append(f"x{j + 1}")
        terms.append("".join(monomial))
    return " [?] ".join(terms)

```

```

@property
def find_fictive_vars(self):
    var_flags = 0
    for i in range(self.anf.shape[0]):
        if self.anf[i]:
            var_flags |= i
    fictive_vars = [i for i in range(self.n) if not (var_flags & (1 <<
i))]
    return fictive_vars if fictive_vars else None

```

```

@njit
def fmt(g, n):
    for i in range(n):
        step = 1 << i
        block_size = step << 1
        for block_start in range(0, 1 << n, block_size):
            for j in range(step):
                a = block_start + step + j
                g[a] ^= g[a - step]
    return g

```

```

@njit
def fwht(arr):
    res = arr.copy()
    n = res.shape[0]
    h = 1
    while h < n:
        for i in range(0, n, h * 2):
            for j in range(i, i + h):
                x = res[j]
                y = res[j + h]
                res[j] = x + y
                res[j + h] = x - y
        h *= 2
    return res

```

```

@njit
def fgbp(size: int, m: int, popcount_table: np.ndarray) -> np.ndarray:
    """Генерирует массив чисел, где вес (popcount) <= m."""
    # Сначала считаем количество подходящих элементов
    count = 0
    for a in range(1, size):
        if popcount_table[a] <= m:
            count += 1
    # Создаём массив нужного размера
    result = np.empty(count, dtype=np.uint32)
    idx = 0
    for a in range(1, size):
        if popcount_table[a] <= m:
            result[idx] = a
            idx += 1
    return result

```

```

@njit
def max_abs_seq(arr: np.ndarray):
    max_val = np.abs(arr[0])
    result = [(arr[0], 0)]
    for i in range(1, arr.shape[0]):
        val = arr[i]
        abs_val = abs(val)
        if abs_val > max_val:
            max_val = abs_val
            result = [(val, i)]
        elif abs_val == max_val:
            result.append((val, i))
    return result

```

```

@njit(parallel=True)
def max_abs_par(arr: np.ndarray):
    max_val = np.abs(arr[0])
    result = [(arr[0], 0)]
    for i in prange(1, arr.shape[0]):
        val = arr[i]
        abs_val = abs(val)

```

```

    if abs_val > max_val:
        max_val = abs_val
        result = [(val, i)]
    elif abs_val == max_val:
        result.append((val, i))
return result

```

@njit

```

def linear_function_bitpacked_seq(a):
    n = a.shape[0]
    bit_len = 1 << n
    num_bytes = (bit_len + 7) // 8
    ax = np.zeros(num_bytes, dtype=np.uint8)
    for i in range(n):
        if a[i]:
            index = 1 << i
            ax[index // 8] |= 1 << (index % 8)
    return ax

```

@njit(parallel=True)

```

def linear_function_bitpacked_par(a):
    n = a.shape[0]
    bit_len = 1 << n
    num_bytes = (bit_len + 7) // 8
    ax = np.zeros(num_bytes, dtype=np.uint8)
    for i in prange(n):
        if a[i]:
            index = 1 << i
            ax[index // 8] |= 1 << (index % 8)
    return ax

```

@njit

```

def int_to_bitarray(x: int, n: int) -> np.ndarray:
    out = np.empty(n, dtype=np.uint8)
    for i in range(n):
        out[n - 1 - i] = (x >> i) & 1
    return out

```

```

@njit
def algebraic_degree_anf(anf):
    max_deg = 0
    for i in range(anf.shape[0]):
        if anf[i]:
            deg = 0
            x = i
            while x:
                x &= x - 1
                deg += 1
            if deg > max_deg:
                max_deg = deg
    return max_deg

```

```

@njit
def upper_deg(w, n):
    deg = 0
    for d in range(1, n + 1):
        low = 1 << (n - d)
        high = (1 << n) - low
        if low <= w <= high:
            deg = d
    return deg

```

```

@njit
def get_degree_candidates(w, n):
    candidates = np.empty(n, dtype=np.uint8)
    i = 0
    for d in range(1, n + 1):
        low = 1 << (n - d)
        high = (1 << n) - low
        if low <= w <= high:
            candidates[i] = d
            i += 1
    return candidates[:i]

```

```

@njit
def binom(n, k):
    if k < 0 or k > n:
        return 0
    if k == 0 or k == n:
        return 1
    res = 1
    for i in range(1, k + 1):
        res = res * (n - i + 1) // i # n(n-1)...(n-k+1) // k!
    return res

```

```

@njit
def generate_bitmask_indices_fast(n, d):
    total = binom(n, d)
    result = np.empty(total, dtype=np.uint32)

    idx = 0
    bits = np.arange(d, dtype=np.uint32)
    while True:
        # Преобразуем позиции в число
        num = 0
        for i in range(d):
            num |= 1 << bits[i]
        result[idx] = num
        idx += 1

        # Найти следующую комбинацию
        for i in range(d - 1, -1, -1):
            if bits[i] != i + n - d:
                break
        else:
            break # всё сгенерировано

        bits[i] += 1
        for j in range(i + 1, d):
            bits[j] = bits[j - 1] + 1

    return result

```

```

@njit
def generate_vectors_fast(n: int, m: int):
    """Генерация всех векторов с весом <= m возвращает( массив np.uint32)."""
    total = 0
    for k in range(1, m + 1):
        total += binom(n, k)
    result = np.empty(total, dtype=np.uint32)
    idx = 0

    for k in range(1, m + 1):
        bitmasks = generate_bitmask_indices_fast(n, k)
        for mask in bitmasks:
            result[idx] = mask
            idx += 1
    return result

```